

Modul Pembelajaran ROS 2 Dasar



1. Pendahuluan

1.1. Apa itu ROS?

ROS, atau Robot Operating System, bukanlah sistem operasi tradisional, melainkan kerangka kerja perangkat lunak berbasis middleware yang dirancang untuk mendukung pengembangan aplikasi robotika. ROS menyediakan berbagai alat, pustaka, dan konvensi yang memudahkan pengembang dalam membangun perangkat lunak robot. ROS awalnya dikembangkan oleh Willow Garage pada tahun 2007 dan kini dikelola oleh Open Robotics. ROS mendukung berbagai platform, termasuk Ubuntu Linux sebagai sistem operasi utama, dengan versi seperti ROS 1 (seperti Kinetic, Melodic) dan ROS 2 (Foxy, Humble) yang menawarkan fitur lebih modern dan performa yang lebih baik.

Komponen Utama ROS

Nodes	Node adalah unit eksekusi dasar di ROS. Setiap node bertanggung jawab untuk tugas tertentu, misalnya membaca data dari sensor atau mengontrol motor.
Topics	Node berkomunikasi melalui <i>topics</i> menggunakan model publisher-subscriber. Misalnya, node sensor mempublikasikan data, dan node lain dapat berlangganan untuk menerima data tersebut.
Services	Selain komunikasi asynchronous dengan <i>topics</i> , ROS juga mendukung komunikasi synchronous menggunakan <i>services</i> , yang memungkinkan request-response antar node.
Messages	<i>Messages</i> adalah struktur data yang digunakan untuk bertukar informasi antar node. Formatnya dapat berupa

	angka, string, atau array.
Bag Files	ROS mendukung perekaman dan pemutaran ulang data melalui format file bernama <i>bag files</i> . Hal ini mempermudah debugging dan analisis data.
Tools	• rviz: Untuk visualisasi data robot. • Gazebo: Untuk simulasi fisika robot dalam lingkungan 3D.

Kelebihan ROS

Open Source	ROS bersifat open-source dan memiliki komunitas aktif yang terus berkontribusi dalam pengembangan perangkat lunak dan pustaka baru.
Modular dan Fleksibel	Komponen-komponen ROS dapat dipecah menjadi modul yang dapat digunakan kembali, memudahkan kolaborasi dan pengembangan.
Dukungan Multiplatform	Meskipun lebih populer di Linux, ROS juga mendukung Windows dan macOS.
Ekosistem yang Luas	ROS memiliki pustaka besar untuk tugas robotika seperti SLAM, navigasi, dan manipulasi objek.
Integrasi dengan Simulasi	ROS terintegrasi dengan alat simulasi seperti Gazebo, memungkinkan

	pengujian tanpa perangkat keras fisik.
--	--

Kekurangan ROS

Kompleksitas	ROS memiliki kurva belajar yang curam, terutama bagi pemula di bidang robotika dan Linux.
Ketergantungan pada Linux	Sebagian besar fitur ROS dirancang untuk Ubuntu Linux, yang mungkin membatasi pengguna yang terbiasa dengan sistem operasi lain.
Performa Real-Time Terbatas	ROS 1 tidak dirancang untuk aplikasi real-time, meskipun ROS 2 telah mengatasi sebagian besar masalah ini.
Fragmentasi Versi	Banyak proyek yang masih menggunakan ROS 1 meskipun ROS 2 menawarkan fitur yang lebih baik, menciptakan fragmentasi dalam ekosistem.

Contoh aplikasi ROS

1. Robot Navigasi
2. Manipulator Robot
3. Drone Otonom
4. Robot Industri
5. Robot Sosial

1.1.1. Mengapa ROS Dibuat

ROS diciptakan untuk menjawab tantangan kompleksitas dalam pengembangan perangkat lunak robotika dan mendorong pengembangan kolaboratif.

Alasan utama ROS dibuat:

Masalah pada Robotika	Solusi ROS
Setiap pengembang membuat sistem dari nol, tidak ada standarisasi	Menyediakan kerangka kerja modular, fleksibel, dan open-source yang bisa digunakan bersama
Sulit membangun perilaku robot kompleks	Menyediakan alat, pustaka, dan konvensi untuk menyederhanakan tugas menciptakan perilaku robot yang kompleks dan kuat
Tidak ada kolaborasi antar peneliti/developer	Diciptakan dari bawah ke atas untuk mendorong pengembangan perangkat lunak robotika kolaboratif seperti dibangun dari beberapa node lalu digabungkan
Kode tertutup dan tidak bisa dibagikan	Open Source
Tidak ada dukungan multibahasa	Menawarkan dukungan multibahasa (C++, Python)

1.1.2. Konsep Middleware

Middleware adalah perangkat lunak yang berfungsi sebagai jembatan antara aplikasi, perangkat lunak, atau sistem yang berbeda dalam suatu arsitektur teknologi informasi. Istilah "*middleware*" pertama kali muncul pada tahun 1968, dan sejak itu, *middleware* telah berkembang menjadi elemen kunci dalam banyak sistem komputer modern. Lalu Apa itu *middleware*? *Middleware* bertindak sebagai perantara yang memungkinkan komunikasi dan pengelolaan data antara berbagai komponen aplikasi yang berjalan di satu atau lebih server.

Middleware adalah lapisan perangkat lunak yang berada di antara sistem operasi dan aplikasi yang berjalan di atasnya. Ia bertindak sebagai perantara yang cerdas, memfasilitasi komunikasi dan manajemen data antar aplikasi yang berbeda. Dengan kata lain, *middleware* memastikan semua bagian sistem dapat berbincang satu sama lain dengan lancar, meskipun mereka "berbicara" menggunakan bahasa pemrograman yang berbeda.

1.1.3. ROS bukan Sistem operasi

Meskipun namanya mengandung kata "sistem operasi," **ROS bukan sistem operasi tradisional** seperti Windows, macOS, atau Linux.

Perbedaan ROS dengan sistem operasi tradisional:

Sistem Operasi Tradisional	ROS (Robot Operating System)
Mengelola hardware (CPU, memory, disk, I/O)	Berjalan di atas sistem operasi (biasanya Ubuntu Linux)
Menyediakan kernel, driver, manajemen proses	Berfungsi sebagai kerangka kerja perangkat lunak berbasis middleware
Contoh: Windows, macOS, Ubuntu	Contoh: ROS 1 (Kinetic, Melodic), ROS 2 (Foxy, Humble, Jazzy)
Bekerja langsung dengan hardware	Menyediakan alat, pustaka, dan konvensi untuk membangun perangkat lunak robot

1.2. Sejarah Singkat ROS

ROS pertama kali dikembangkan di **Willow Garage** pada **tahun 2007** di Stanford University, melibatkan banyak peneliti pakar robotika. ROS 0.4 dirilis pada awal 2009, dan **ROS 1.0 dirilis pada tahun 2010**. Sejak itu, ROS menjadi standar dalam dunia robotika dengan komunitas global yang aktif.

1.2.1. ROS 1

Aspek	Detail
Tahun rilish	2010 (ROS 1.0)
Arsitektur	Berbasis roscore sebagai pusat komunikasi
Middleware	TCPROS dan UDPROS (protokol kustom ROS)

Build System	Catkin
Platform	Linux (Ubuntu) dan macOS
Bahasa	C++ 03 dan Python 2
Status	End of Life (EOL) pada 2025 untuk Noetic

Diantarannya versi ROS1:

Distro	Tahun Rilis	Status
Indigo	2014	EOL (2019)
Kinetic	2016	EOL (2021)
Melodic	2018	EOL (2023)
Noetic	2020	EOL (2025) - versi LTS terakhir ROS 1

1.2.2. ROS 2

Aspek	Detail
Tahun mulai	2014 (dimulai), 2018 (ROS 2 LTS pertama)
Arsitektur	Dibangun ulang dengan arsitektur modern
Middleware	DDS (Data Distribution Service) - standar industri untuk komunikasi real-time
Build System	Colcon (universal build tool)
Platform	Linux, Windows 10+, macOS
Bahasa	C++ 11/14 dan Python 3.5+
Fitur	Real-time, sistem terdistribusi, QoS (Quality

	of Service), siap produksi
--	----------------------------

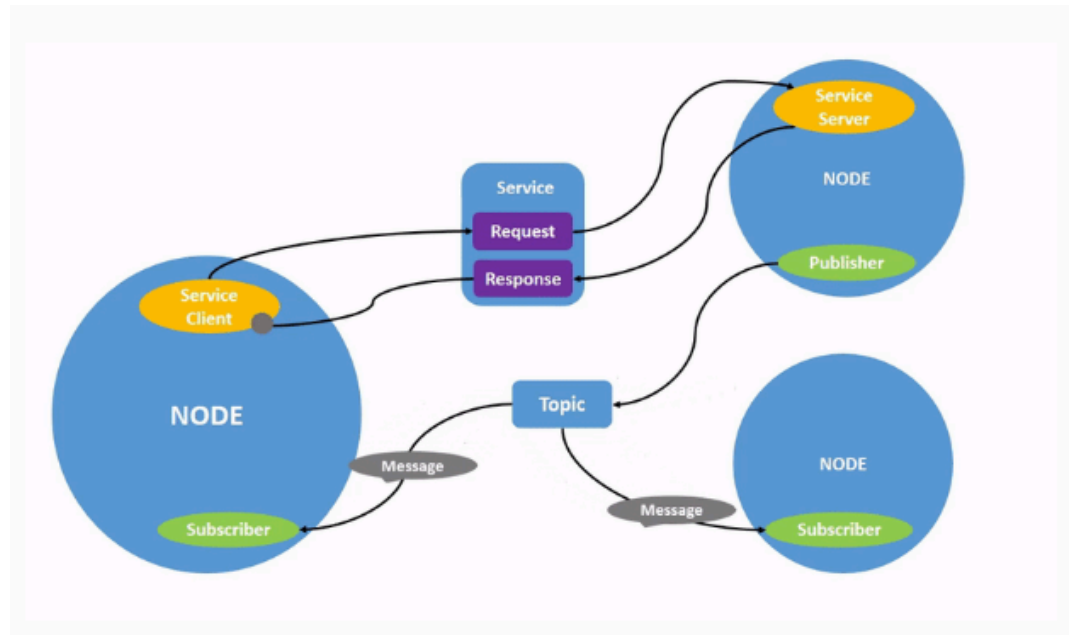
Diantarannya versi ROS2:

Distro	Tahun Rilis	Basis Ubuntu
Foxy	2020	Ubuntu 20.04
Humble	2022	Ubuntu 22.04
Iron	2023	Ubuntu 22.04
Jazzy	2024	Ubuntu 24.04

2. Arsitektur ROS 2

2.1. Konsep Node

Setiap node di ROS harus bertanggung jawab untuk tujuan tunggal, modular, misalnya mengendalikan motor roda atau menerbitkan data sensor dari laser range-finder. Setiap node dapat mengirim dan menerima data dari node lain melalui topik, layanan, tindakan, atau parameter.

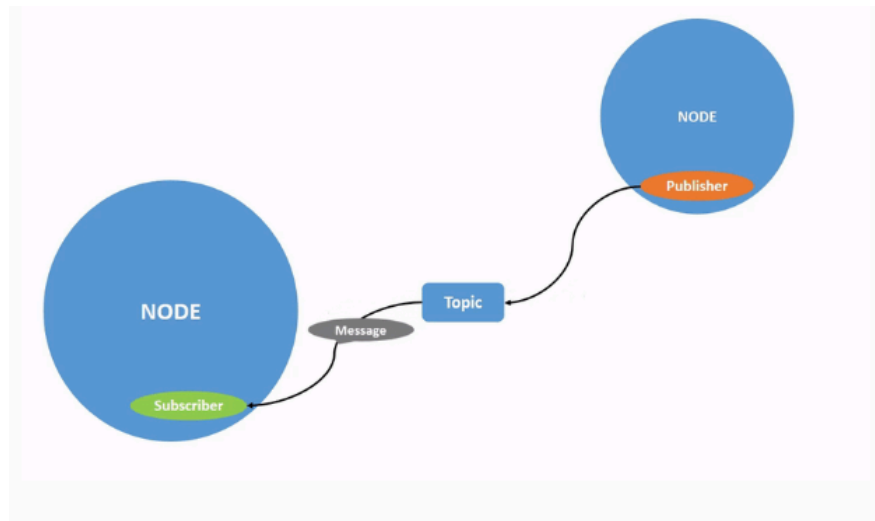


[Source gambar](#)

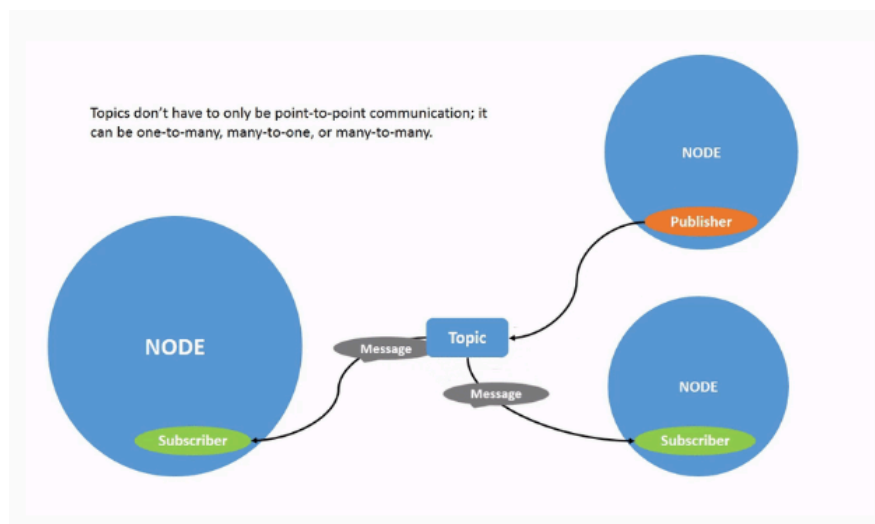
Sistem robot penuh terdiri dari banyak node yang bekerja dalam konser. Dalam ROS 2, satu executable (program C ++, program Python, dll) dapat berisi satu atau lebih node.

2.2. Konsep Topic

ROS 2 memecah sistem yang kompleks menjadi banyak node modular. Topik adalah elemen penting dari grafik ROS yang bertindak sebagai bus untuk node untuk bertukar pesan.



Sebuah node dapat mempublikasikan data ke sejumlah topik dan secara bersamaan memiliki langganan ke sejumlah topik.



Topic adalah cara utama dimana data berpindah antara node-node dan antara bagian sistem yang berbeda.

[Source gambar](#)

Coba gunakan kode dibawah ini

[Source Code C++](#)

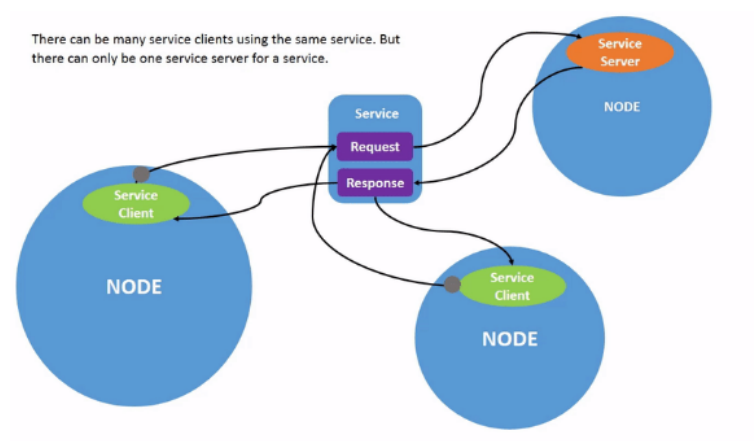
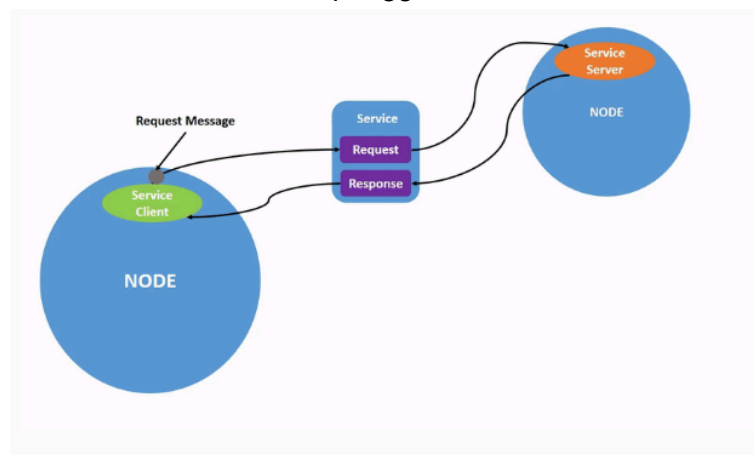
[Source Code Python](#)

2.3. Komunikasi Lain dalam ROS

Selain Publish dan Subscribe ada juga Service dan juga Action walaupun jarang dipakai tidak sesering Publish dan Subscribe.

2.3.1. Service

Service adalah metode komunikasi lain untuk node dalam grafik ROS. Layanan didasarkan pada model panggilan-dan-respons versus model penerbit-pelanggan topik. Sementara topik memungkinkan node untuk berlangganan aliran data dan mendapatkan pembaruan terus-menerus, layanan hanya menyediakan data ketika mereka secara khusus dipanggil oleh klien.



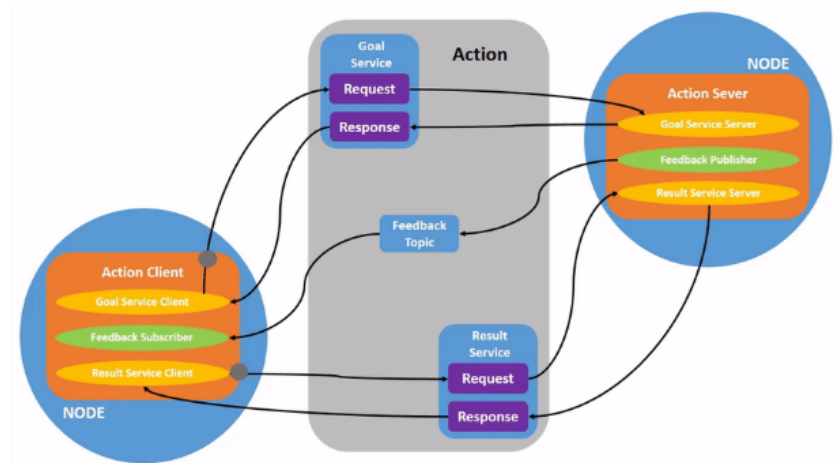
[Source gambar](#)

2.3.2. Action

Action adalah salah satu jenis komunikasi di ROS 2 dan ditujukan untuk tugas-tugas jangka panjang. Mereka terdiri dari tiga bagian: tujuan, umpan balik, dan hasil.

Action dibangun di atas topik dan layanan. Fungsionalitas mereka mirip dengan layanan, kecuali tindakan yang preemptable (Anda dapat membatalkan mereka saat mengeksekusi). Mereka juga memberikan umpan balik yang mantap, sebagai lawan dari layanan yang mengembalikan satu respons.

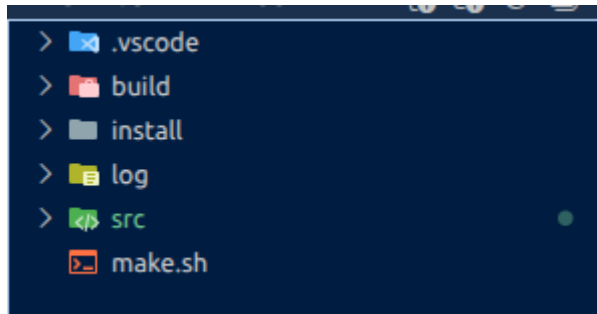
Action menggunakan model client-server, mirip dengan model penerbit-pelanggan (dijelaskan dalam topik tutorial). Sebuah "action client" node mengirimkan tujuan untuk "fungsi server" node yang mengakui tujuan dan mengembalikan aliran umpan balik dan hasil.



[Source gambar](#)

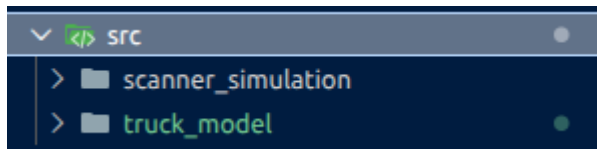
3. Workspace dan Package

Struktur workspace



Isi dari src adalah package

dibawah adalah contohnya:



Membuat Workspace

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws  
colcon build
```

Membuat Package

Python

```
ros2 pkg create my_package --build-type ament_python
```

C++

```
ros2 pkg create my_package --build-type ament_cmake
```

CLI Dasar ROS2

[ros2 topic list] digunakan untuk melihat list topic yang berjalan

[ros2 topic echo <nama topic>] digunakan untuk log topic

[ros2 node list] digunakan untuk melihat node yang berjalan

[ros2 topic hz <nama topic>] digunakan untuk melihat Hz topic dan melihat apakah topic mengirim data

[rqt] Digunakan untuk menjalankan aplikasi RQT yang menyediakan berbagai plugin GUI untuk monitoring dan debugging sistem ROS 2.

[rqt_graph] Digunakan untuk memvisualisasikan hubungan komunikasi antar node dan topic dalam bentuk ROS Graph.
[rviz2] Digunakan untuk memvisualisasikan data robot, sensor, peta, dan TF secara interaktif.

Pemahaman CMake

1. Perintah dasar dan ament

- `cmake_minimum_required(VERSION 3.8)`: Menetapkan versi minimum CMake (3.8 adalah standar untuk banyak distribusi ROS 2).
- `project(<nama_paket>)`: Mendefinisikan nama paket sesuai dengan `package.xml`.
- `find_package(ament_cmake REQUIRED)`: Perintah paling krusial untuk memuat CMake macros milik ROS 2.
- `ament_package()`: Perintah terakhir yang mendaftarkan paket ke sistem build ament, harus diletakkan di akhir file.

2. Manajemen Dependensi

- `find_package(<dependensi> REQUIRED)`: Mencari dependensi eksternal atau paket ROS 2 lainnya, seperti `rclcpp` (untuk C++), `std_msgs` (pesan standar), atau `sensor_msgs`.
- `find_package(rosidl_default_generators REQUIRED)`: Diperlukan jika paket membuat pesan/layanan kustom.

3. Build Executables dan Libraries

- `add_executable(<nama_node> src/<file_source>.cpp)`: Membuat file eksekusi (node) dari file sumber C++.
- `add_library(<nama_lib> src/<file_source>.cpp)`: Membuat library jika kode akan digunakan kembali.
- `target_include_directories(<target> PUBLIC include)`: Menentukan direktori header (jika ada folder `include`).
- `ament_target_dependencies(<target> <dep1> <dep2>)`: Menghubungkan dependensi yang ditemukan via `find_package` ke executable/library target. Ini otomatis menangani *include directories* dan *linking*.

4. Instalasi dan Ekspor

- `install(TARGETS ... DESTINATION lib/${PROJECT_NAME})`: Menginstal file executable agar bisa dijalankan oleh `ros2 run`.
- `install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})`: Menginstal folder tambahan seperti *launch file* atau konfigurasi agar dapat ditemukan oleh ROS 2.
- `ament_export_dependencies()`: Mengekspor dependensi agar paket lain yang bergantung pada paket ini otomatis mendapatkan dependensi tersebut.

Dibawah adalah contoh **CMakeLists.txt**:

```
cmake_minimum_required(VERSION 3.8)
project(my_robot_controller)

# Mencari dependensi
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

# Membuat executable
add_executable(my_node src/my_node.cpp)
ament_target_dependencies(my_node rclcpp std_msgs)

# Instalasi
install(TARGETS
  my_node
  DESTINATION lib/${PROJECT_NAME}
)

ament_package()
```

4. Penugasan Pub Sub

PENUGASAN 1

Terdapat 3 package:

Package A

Memiliki node **ProducerA**:

Tugas:

- Mengirim angka increment mulai dari 1.
- Publish ke topic `/data_a`.
- Saat menerima sinyal STOP dari Package B, berhenti mengirim data.

- Saat menerima sinyal START dari Package B, melanjutkan publish.

Package B

Memiliki node **Coordinator**.

Tugas:

- Menerima data dari A.
- Menghitung jumlah data yang diterima.

Jika jumlah data mencapai N:

STOP A

START C

Kemudian:

- Berhenti menerima data dari A.
- Menunggu data dari C.

Jika jumlah data dari C mencapai M:

STOP C

STOP A

Siklus berulang terus.

Package C

Memiliki node **ProducerC**.

Tugas:

- Dalam kondisi awal tidak aktif.
- Saat menerima START dari B:
 - Mengirim data increment ke topic **/data_c**.
- Saat menerima STOP dari B:
 - Berhenti mengirim data.

Gunakan topik dibawah atau terserah

/data_a

/data_c

/control_a

/control_c

PENUGASAN 2

Menjalankan turtlesim

install turtlesim

`sudo apt update`

`sudo apt install ros-<distro-ros>-turtlesim`

Cek apakah sudah terinstall:

```
ros2 pkg executables turtlesim
```

Jalankan turtlesim:

```
ros2 run turtlesim turtlesim_node
```

Gunakan domain yang sama antar komputer

```
export ROS_DOMAIN_ID=10
```

cek topic dan node

```
ros2 node list => /turtlesim
```

```
ros2 topic list => topic dari turtlesim
```

pertama spawn robot anda pada node admin

cek service yang tersedia cari /spawn

```
ros2 service list
```

cek tipe service dan Cek isi request service

```
ros2 service type /spawn
```

dan

```
ros2 interface show turtlesim/srv/Spawn
```

contoh memanggil service

```
ros2 service call /spawn
```

```
turtlesim/srv/Spawn "{x: 5.0, y: 5.0, theta: 0.0,  
name: 'robot_wildan'}"
```

coba jalan node teleop key

```
ros2 run turtlesim turtle_teleop_key
```

tetapi disini tidak bisa menyambung karena topic berbeda disini harus dilakukan remapping topic dengan CLI dibawah

```
ros2 run <package_name> <executable_name>  
--ros-args --remap  
<nama_topik_lama>:=<nama_topik_baru>
```

samakan dengan topic anda

Buat node tugasnya untuk mengontrol turtle menggunakan keyboard pada laptop anda dimana ada

tombol maju, mundur, belok kiri dan kanan dan stop.

5. Launch dan YAML

Karena hampir semua proyek ROS menggunakan keduanya.

Launch

Tanpa launch untuk menjalankan ROS2 dapat memakan banyak waktu dikarenakan 1 terminal menjalankan 1 node, jika menggunakan launch dimana 1 launch dapat menjalankan lebih dari 1 node maka dapat menghemat terminal dan mempersingkat waktu.

contoh:

- **Tanpa launch:**
ros2 run publisher
ros2 run subscriber
- **Dengan launch:**
ros2 launch my_pkg demo.launch.py

pada project sebelumnya pada turtlesim anda bisa membuat launch yang sebelumnya memerlukan lebih dari 1 terminal kini bisa dijalankan dengan 1 terminal saja langkah langkahnya dibawah

1. buat folder **Build** pada package ros2_turtlesim
2. tambahkan file <nama_file [.launch.py](#)>
3. tambahkan kode sederhana dibawah kode akan menjalankan executable teleop anda

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():

    teleop_turtle_node = Node(
        package='ros2_turtlesim',
        executable='teleop_turtle',
        name='teleop_turtle',
        output='screen',
        remappings=[('/turtle1/cmd_vel',
                    '/turtle2/cmd_vel')]
    )

    return LaunchDescription([
        teleop_turtle_node
    ])

```

4. tambahkan service call spawn ini dibawah:

a.

```

from launch.actions import ExecuteProcess

spawn_turtle_node = ExecuteProcess(cmd=[
    'ros2', 'service', 'call',
    '/spawn',
    'turtlesim/srv/Spawn',
    "{x: 5.0, y: 5.0, theta: 0.0, name: 'robot_wildan'}"
],
    output='screen'
)

```

b.

5. Lalu coba anda tambahkan node teleop anda sendiri jangan lupa lihat executable pada cmakeLists anda, lalu tambahkan service dibawah

YAML

Yaml digunakan untuk mempermudah konfigurasi pada kode agar kode jika memiliki nilai yang ingin diubah tidak membuild ulang node atau kode ROS2

contoh:

```

publisher:
  ros__parameters:
    rate: 10

```

6. LAIN LAIN

Intermediate

- Service
- Action
- Parameter
- QoS
- TF2

Advanced

- Lifecycle Node
- Navigation 2
- SLAM
- MoveIt
- Multi Robot
- DDS dan Middleware

referensi

<https://docs.ros.org/en/humble/index.html>

<https://comp-eng.binus.ac.id/2024/12/09/apa-itu-ros-sistem-operasi-untuk-robot-modern/>

<https://www.codepolitan.com/blog/apa-itu-middleware-pengertian-contoh-dan-fungsinya/>